

Smart Collision Detection and Automatic Emergency Braking System

Meet Shinde

KPIT APEX LAB

COEP Technological University
prashantsm24.extc@coeptech.ac.in

Siddhant Wankhede

KPIT APEX LAB

COEP Technological University
raj कुमारws24.extc@coeptech.ac.in

Vedant Yadav

KPIT APEX LAB

COEP Technological University
hemantyv24.extc@coeptech.ac.in

Mentor - Jayanth Subramanian

KPIT Technologies

Jayanth.Subramanian@kpit.com

Abstract—This paper presents the design, implementation, and validation of a low-cost, multi-node Autonomous Emergency Braking (AEB) system built on a distributed Controller Area Network (CAN) architecture. The system integrates four embedded nodes — an Arduino Nano transmitting joystick commands over a 2.4 GHz RF link, an Arduino Uno bridging RF input onto the CAN bus, an Arduino Mega 2560 acting as the perception Electronic Control Unit (ECU) through LIDAR and ultrasonic sensor fusion, and an STM32 Nucleo-64 (F446RE) motor controller driving a DC motor through an L298N H-bridge. Beyond baseline collision avoidance, the system incorporates an EEPROM-based Event Data Recorder (EDR) that captures pre- and post-trigger telemetry analogous to automotive black-box systems, and a Proportional-Integral-Derivative (PID) based Adaptive Cruise Control (ACC) loop that replaces discrete braking zones with continuous distance-regulated control. A CAN heartbeat deadman-switch mechanism provides fail-safe behavior in the event of node or communication failure. The architecture, firmware design, fusion logic, fail-safe mechanisms, and diagnostic layer are described in detail, along with the rationale behind key design decisions and the limitations of the present implementation.

Index Terms—Autonomous Emergency Braking, Controller Area Network, Embedded Systems, Sensor Fusion, Event Data Recorder, Adaptive Cruise Control, STM32, Arduino, Fail-Safe Systems.

I. INTRODUCTION

Autonomous Emergency Braking (AEB) has become one of the most widely deployed Advanced Driver Assistance System (ADAS) features in modern passenger vehicles, credited with significant reductions in rear-end collision rates. Commercial AEB implementations typically rely on radar, camera, or LIDAR sensing fused with a centralized or distributed Electronic Control Unit (ECU) network communicating over an automotive-grade Controller Area Network (CAN) bus. While these production systems are well documented at a high level, the underlying multi-node communication architecture, fail-safe design, and diagnostic infrastructure are rarely accessible for hands-on study at the student or hobbyist level.

This project set out to construct a scaled, low-cost analogue of such a system using commodity microcontroller hardware,

with the explicit goal of reproducing — rather than merely simulating — the architectural patterns found in real automotive safety systems: distributed nodes communicating over a shared CAN bus, sensor fusion for robust perception, a multi-stage braking response, a hardware fail-safe independent of the main control logic, and auxiliary systems for diagnostics and incident logging.

The resulting platform consists of four nodes on a shared 500 kbps CAN bus, identified by three primary CAN identifiers: 0x100 for joystick commands, 0x050 for the heartbeat fail-safe signal, and 0x200 for AEB telemetry. Beyond the core AEB function, the system was extended in two directions that mirror real automotive systems: an Event Data Recorder (EDR) that preserves a window of telemetry around a braking event, similar in spirit to an automotive black box, and an Adaptive Cruise Control (ACC) loop that replaces the original discrete braking zones with continuous PID-based distance regulation.

The remainder of this paper is organized as follows. Section II reviews related work and positions this project relative to existing AEB literature and embedded automotive safety practice. Section III describes the overall system architecture and CAN bus topology. Section IV details the perception and sensor fusion subsystem. Section V describes the core AEB braking logic and the heartbeat-based fail-safe mechanism. Section VI presents the Adaptive Cruise Control subsystem. Section VII presents the Event Data Recorder. Section VIII discusses the diagnostic layer. Section IX presents results and discussion, Section X discusses limitations and future work, and Section XI concludes the paper.

II. RELATED WORK

Production AEB systems in passenger vehicles, such as those evaluated by independent safety bodies, typically combine forward-facing radar or camera sensing with brake-by-wire actuation and are validated against standardized test protocols involving stationary and moving target scenarios. These systems operate over automotive CAN or CAN-FD networks

governed by standards such as ISO 11898 for the physical and data link layers and ISO 14229 (Unified Diagnostic Services, UDS) for diagnostic communication [1], [2].

Academic and hobbyist projects have explored individual elements of this architecture in isolation — ultrasonic or LIDAR-based obstacle braking on a single microcontroller, CAN bus communication demonstrations between two nodes, or EEPROM-based data logging for embedded systems coursework. However, comparatively few openly documented projects integrate sensor fusion, a multi-node CAN architecture, a hardware-independent fail-safe, adaptive cruise control, and black-box-style event logging into a single cohesive platform built entirely on low-cost development boards. This project’s contribution is primarily one of systems integration: demonstrating that these distinct automotive safety patterns can be reproduced faithfully, and made to interoperate correctly, using an Arduino and STM32 based hardware stack.

III. SYSTEM ARCHITECTURE

The system is organized as four independent nodes communicating over a shared CAN bus operating at 500 kbps, using MCP2515 CAN controller modules [3] for the Arduino-based nodes and the integrated CAN peripheral of the STM32 for the motor controller node. Table I summarizes the CAN identifier allocation and Fig. 1 illustrates the overall data flow between nodes.

A. Node Overview

- **Arduino Nano (RF Transmitter):** Reads joystick analog input and transmits commands wirelessly over a 2.4 GHz NRF24L01 link to the bridge node.
- **Arduino Uno (RF-to-CAN Bridge):** Receives RF joystick packets and re-publishes them onto the CAN bus under identifier 0x100, acting as the gateway between the wireless control link and the wired CAN network.
- **Arduino Mega 2560 (Perception ECU):** Performs sensor fusion between a LIDAR-Lite v3HP rangefinder [4] and an HC-SR04 ultrasonic sensor, computes a fused distance estimate, and broadcasts AEB telemetry under identifier 0x200. Also originates the heartbeat signal under identifier 0x050. Hosts the EEPROM-based Event Data Recorder.
- **STM32 Nucleo-64 (F446RE) (Motor Controller):** Consumes CAN traffic from the joystick bridge and perception ECU — including the heartbeat originated by the perception node — arbitrates between manual driver input and AEB/ACC intervention, and drives a DC motor through an L298N H-bridge [5], [6]. Originates no CAN traffic of its own in the current firmware revision.

B. Design Rationale

A distributed architecture was chosen over a single-microcontroller implementation deliberately, in order to mirror the separation of concerns found in real vehicle ECU networks: perception, actuation, and human input are handled by physically and logically separate nodes, communicating only

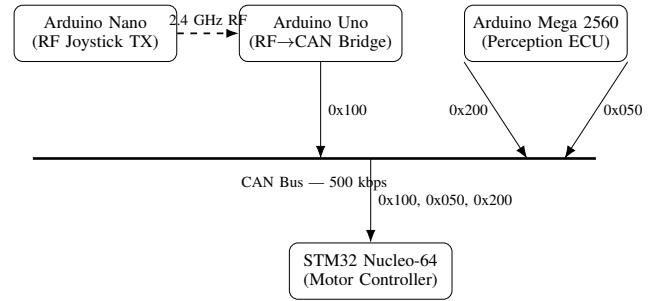


Fig. 1: System architecture and data flow between the four CAN nodes. The Arduino Uno broadcasts onto the shared CAN bus under identifier 0x100; the Arduino Mega 2560 broadcasts under both 0x200 (telemetry) and 0x050 (heartbeat). The STM32 motor controller consumes all three identifiers and originates no CAN traffic of its own in the current firmware revision.

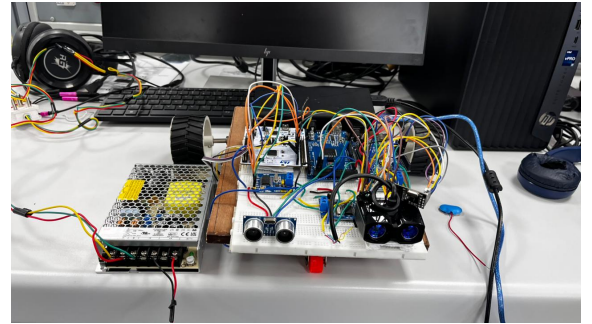


Fig. 2: Side-angle photograph of the assembled hardware platform, showing the breadboard-mounted control nodes, power supply, and drive wheels.

through well-defined CAN messages. This separation also has a practical safety benefit reproduced from real systems — a fault in the perception node (e.g., a sensor freeze) does not directly corrupt the motor controller’s state, since the motor controller only acts on validated, timestamped CAN traffic and independently enforces the heartbeat fail-safe described in Section V.

During development, several hardware-level conflicts had to be resolved as a direct consequence of this distributed design, including SPI bus contention between the NRF24L01 and MCP2515 modules sharing a bus on the same node, and chip-select pin conflicts between the MCP2515 module and an SD card breakout on the perception node. These were resolved through careful chip-select pin reassignment and bus arbitration in firmware.

IV. PERCEPTION AND SENSOR FUSION

Obstacle perception is performed entirely on the Arduino Mega 2560, which fuses readings from a LIDAR-Lite v3HP time-of-flight rangefinder and an HC-SR04 ultrasonic sensor before broadcasting the result onto the CAN bus.

TABLE I: CAN Identifier Allocation

CAN ID	Source Node	Purpose
0x100	Arduino Uno (Bridge)	Joystick throttle/steering command relayed from RF link
0x050	Arduino Mega (Perception)	Heartbeat signal for deadman-switch fail-safe
0x200	Arduino Mega (Perception)	Fused obstacle distance and AEB zone telemetry

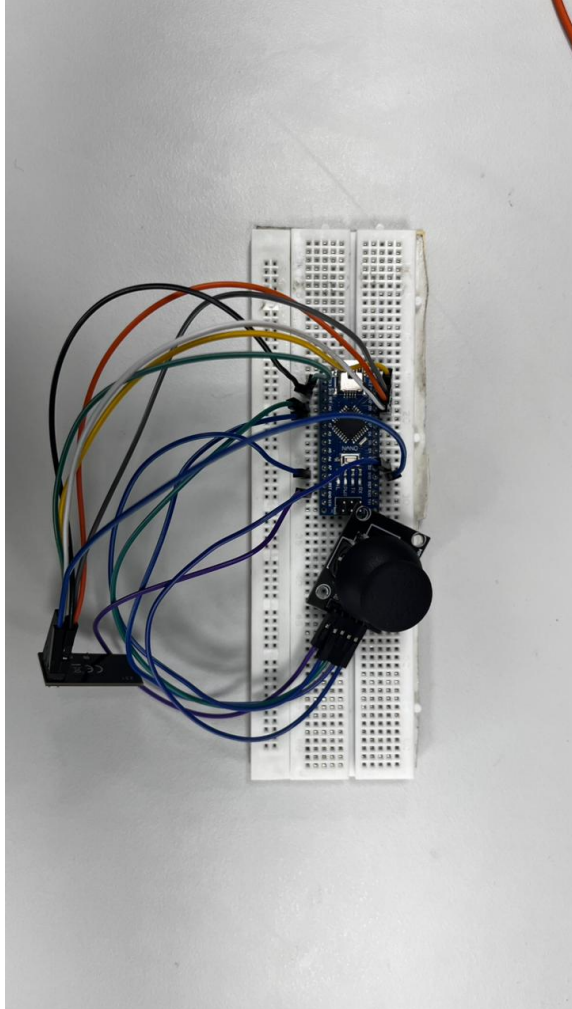


Fig. 3: Arduino Nano transmitter node, showing the HW504 joystick and nRF24L01 radio module wired on the breadboard.

A. Rationale for Dual-Sensor Fusion

Neither sensor alone is fully reliable across all operating conditions. The LIDAR-Lite v3HP offers higher precision and a faster effective sample rate but can produce erroneous readings against highly reflective, transparent, or angled low-reflectivity surfaces. The HC-SR04 ultrasonic sensor is comparatively robust to surface reflectivity but has a wider beam divergence, lower precision, and a slower response time. Fusing both sensors allows the system to cross-validate readings and fall back on the more trustworthy sensor when the two diverge significantly.

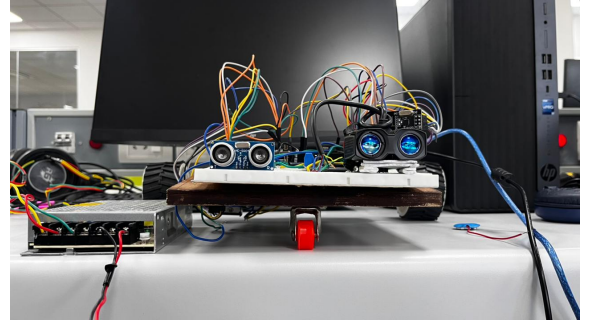


Fig. 4: Close-up of the LIDAR-Lite v3HP and HC-SR04 ultrasonic sensor mount on the perception node.

B. Zone-Based Weighted Fusion

The fusion algorithm computes a weighted average of the two sensor readings, with weights adjusted dynamically based on the distance regime (zone) the readings fall into. At closer ranges, where the LIDAR sensor's precision advantage matters most for braking decisions, the LIDAR reading is weighted more heavily; at longer ranges, where ultrasonic readings remain reliable and LIDAR confidence can degrade for certain target types, the weighting is more balanced. Both raw sensor streams are passed through a moving average filter prior to fusion to suppress sample-to-sample noise and transient dropouts.

An integer-overflow defect was identified during development in the fusion math, where intermediate weighted-sum calculations using 16-bit integer types could overflow under certain combinations of raw readings; this was resolved by widening the relevant intermediate variables to 32-bit signed integers.

C. Telemetry Broadcast

The fused distance estimate, along with the current AEB zone classification, is packed into a CAN frame and broadcast under identifier 0x200 at a fixed interval. The motor controller node on the STM32 treats this frame as the authoritative perception input for both the AEB and ACC subsystems described in Sections V and VI.

V. AEB LOGIC AND FAIL-SAFE DESIGN

A. Multi-Stage Braking Zones

The baseline AEB behavior, prior to the introduction of the PID-based ACC loop described in Section VI, classifies the fused obstacle distance into three discrete zones, each commanding a different motor response on the STM32 motor controller node:

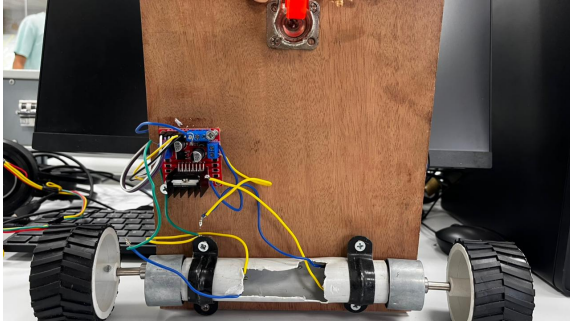


Fig. 5: L298N H-bridge driver and DC motor wiring used for steering/drive actuation on the motor controller node.

- **Cruise Zone:** Obstacle distance is large enough that no intervention is required; the motor controller passes through the driver's joystick command unmodified.
- **Decel Zone:** Obstacle distance has fallen below a deceleration threshold; the motor controller proportionally reduces commanded motor power regardless of driver throttle input.
- **Emergency Zone:** Obstacle distance has fallen below a critical threshold; the motor controller commands active braking using reverse back-EMF braking (briefly reversing polarity through the L298N H-bridge) to bring the motor to a stop more rapidly than coasting alone would allow.

This staged approach reflects the same general philosophy used in production AEB calibration, where escalating levels of intervention are preferred over a single binary braking trigger, in order to balance false-positive intervention against stopping performance.

B. CAN Heartbeat Deadman-Switch Fail-Safe

Independent of the AEB zone logic, the motor controller node implements a heartbeat-based deadman-switch mechanism. The STM32 expects to receive a heartbeat CAN frame under identifier 0x050, originated by the Arduino Mega perception node, at a regular interval; if no heartbeat is received within an 80 ms timeout window, the motor controller treats this as a communication or node failure and immediately enters a hardware lockout state, cutting motor drive output regardless of any other commanded state.

Recovery from the lockout state is intentionally not automatic. Exit from the lockout requires a debounced physical button press on pin PC13 of the Nucleo board, ensuring that a transient fault cannot silently clear itself and resume motor operation without a deliberate human action — a design choice analogous to fault-latching behavior in automotive safety-critical ECUs, where certain fault classes require an explicit reset rather than self-clearing once the triggering condition disappears.

C. UDS-Style Diagnostics

A lightweight diagnostic layer modeled on ISO 14229 (Unified Diagnostic Services) was implemented to allow read

access to internal motor controller state — such as current AEB zone, last fused distance, and heartbeat status — over the CAN bus using a request/response service identifier pattern similar to standard UDS read-data-by-identifier services. This is discussed further in Section VIII.

VI. ADAPTIVE CRUISE CONTROL

The most recent extension to the motor control firmware replaces the discrete Decel/Emergency zone thresholds described in Section V with a continuous closed-loop controller targeting a fixed following gap, implementing a simplified form of Adaptive Cruise Control (ACC).

A. Control Objective

The ACC loop targets a 300 mm following gap between the vehicle and the detected obstacle ahead, using the fused distance telemetry broadcast on CAN ID 0x200 as its feedback signal. Rather than simply commanding full stop or full proportional decel as in the zone-based approach, the controller continuously adjusts motor power to drive the error between the target gap and the actual fused distance toward zero.

B. PID Controller Design

The controller is implemented as a discrete-time PID loop executing on a fixed 20 ms schedule on the STM32. The control output $u(t)$ is computed from the following standard form:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1)$$

where $e(t)$ is the error between the 300 mm target gap and the current fused distance reading. Two refinements were made to the basic PID form during tuning:

- **Derivative low-pass filtering:** The raw derivative term is highly sensitive to sensor noise, particularly given the moving-average-filtered but still discretely-sampled fusion input. A first-order low-pass filter is applied to the derivative term before it is added to the control output, smoothing the controller's response to sensor jitter without materially slowing its reaction to genuine distance changes.
- **Integral anti-windup, later removed:** An integral anti-windup clamp was initially implemented to prevent the integral term from accumulating unbounded error during periods of actuator saturation. During tuning, the integral term was ultimately removed from the controller entirely, leaving a PD-dominant control law; the motivating debugging history for this decision is discussed in Appendix D.

C. Loop Scheduling and Interaction with the Fail-Safe Layer

The 20 ms ACC control schedule runs independently of, and is strictly subordinate to, the heartbeat deadman-switch lockout described in Section V-B — a missed heartbeat immediately overrides any ACC-commanded motor output. The ACC loop also coexists with the multi-stage AEB zones: the Emergency Zone back-EMF braking response described in Section V-A continues to take precedence over the PID output for

obstacle distances below the emergency threshold, so that the continuous ACC regulation handles the steady-state following-gap problem while the discrete emergency stage still provides a hard floor of protection at very short range.

Implementation status: the motor controller firmware verified at the time of writing implements the Decel-zone response as a linear interpolation ramp on commanded PWM duty cycle rather than the full PID form of Eq. 1. The PID design above reflects the target control law; convergence of the deployed firmware toward it is tracked as ongoing work in Appendix D.

VII. EVENT DATA RECORDER

To provide post-incident traceability analogous to an automotive black box, the perception node (Arduino Mega 2560) was extended with an EEPROM-based Event Data Recorder (EDR).

A. Trigger Condition

The EDR begins capturing a recording buffer whenever the fused obstacle distance falls below a 250 mm trigger threshold, on the premise that any approach this close to an obstacle is operationally significant regardless of which AEB zone or ACC state the system was in at the time.

B. Capture Format

Each triggered event captures 21 frames of telemetry sampled at 20 Hz, written into a 254-byte region of the Mega's onboard EEPROM. Each frame records the fused distance, sensor zone/state, and a timestamp offset relative to the trigger, giving a recording window that spans roughly one second on either side of the triggering moment, in the same spirit as the pre- and post-crash data windows captured by commercial EDR units.

C. Interaction with the Watchdog Timer

EEPROM write operations on the AVR-based Mega are comparatively slow relative to the rest of the firmware's loop timing, and a naive blocking write during a capture event risked starving other time-critical tasks long enough to trip the node's watchdog timer. To address this, the EDR write routine injects heartbeat-preserving service calls during its write window, ensuring that downstream consumers of node liveness signals (and the watchdog itself) are serviced even while a multi-frame EEPROM write is in progress.

VIII. DIAGNOSTIC LAYER

A diagnostic service layer, modeled on the structure of ISO 14229 (UDS), was added on top of the CAN bus to support runtime inspection of system state without requiring a physical debugger connection to each node.

Supported diagnostic reads include the current fused obstacle distance, the active AEB zone or ACC state, heartbeat/lockout status, and the most recent EDR trigger timestamp. Requests follow a request/response service-identifier pattern: a diagnostic request frame addressed to a specific node prompts a corresponding response frame containing the requested data, allowing the system's internal state to be polled live from

a CAN bus analyzer or a simple host-side script during debugging and demonstration.

Implementation status: this diagnostic layer describes the target design for runtime introspection. Explicit request/response service-identifier handling has not yet been added to the firmware verified at the time of writing; see Appendix D for further discussion.

IX. RESULTS AND DISCUSSION

A. Qualitative System Behavior

The integrated system was exercised under manual joystick control with obstacles placed at varying distances to qualitatively validate each subsystem in turn: correct CAN frame routing and identifier filtering across all four nodes, correct zone transitions in the baseline AEB logic, correct heartbeat lockout and recovery via the PC13 button, and correct EDR triggering and EEPROM capture below the 250 mm threshold.

Across repeated bench trials, all four subsystems behaved consistently with their design intent: CAN frames were correctly filtered by identifier at each receiving node with no observed cross-talk between the 0x100, 0x050, and 0x200 traffic; the AEB zone state machine transitioned cleanly between CRUISE, DECEL, and EMERG as the fused obstacle distance was varied by hand; severing the CAN-H line or holding the perception node in reset reliably triggered the hardware lockout, which could only be cleared by a PC13 button press of at least the intended debounce duration; and EDR capture triggered correctly on emergency-zone entry, with the resulting EEPROM log retrievable in full over the serial interface after each test. These bench observations were qualitative rather than instrumented, and are consistent with, though not a substitute for, the timestamped verification procedures specified in Appendix C.

B. Quantitative Results

Formal instrumented timing measurement — using the CAN bus analyzer and repeated-trial procedure specified in Appendix C — has not yet been carried out on this system. Table II therefore reports theoretical design-target values, derived directly from the firmware constants and control logic documented in Appendix B, rather than measured data. Qualitative bench observation of system behavior (Section IX-A) was consistent with these targets in every trial conducted — the lockout was never observed to engage later than expected relative to a heartbeat loss, and EDR logs consistently contained the full 21-frame window — but this consistency was observed by eye and stopwatch rather than logged with instrumentation, and so is reported narratively rather than as a verified figure in the table below. Obtaining instrumented measurements against this table is the immediate next step identified in Appendix D.

X. LIMITATIONS AND FUTURE WORK

The present system, while functionally representative of the automotive patterns it draws from, has several limitations relative to production-grade implementations.

TABLE II: Theoretical Design-Target Performance Metrics (Instrumented Measurement Pending)

Metric	Design Target (Theoretical)	Basis
Heartbeat timeout-to-lockout latency	≤ 80 ms	AEB_WATCHDOG_MS constant, Appendix B.4
ACC / Decel-zone speed regulation range	30%–70% PWM duty cycle	Linear ramp formula, Appendix B.4
EDR capture window	21 frames @ 20 Hz (≈ 1.05 s span)	EDR_TOTAL_FRAMES, EDR_SAMPLE_MS, Appendix B.3
Emergency zone stopping distance	Not derivable from firmware alone	Depends on vehicle mass, motor torque, and surface friction

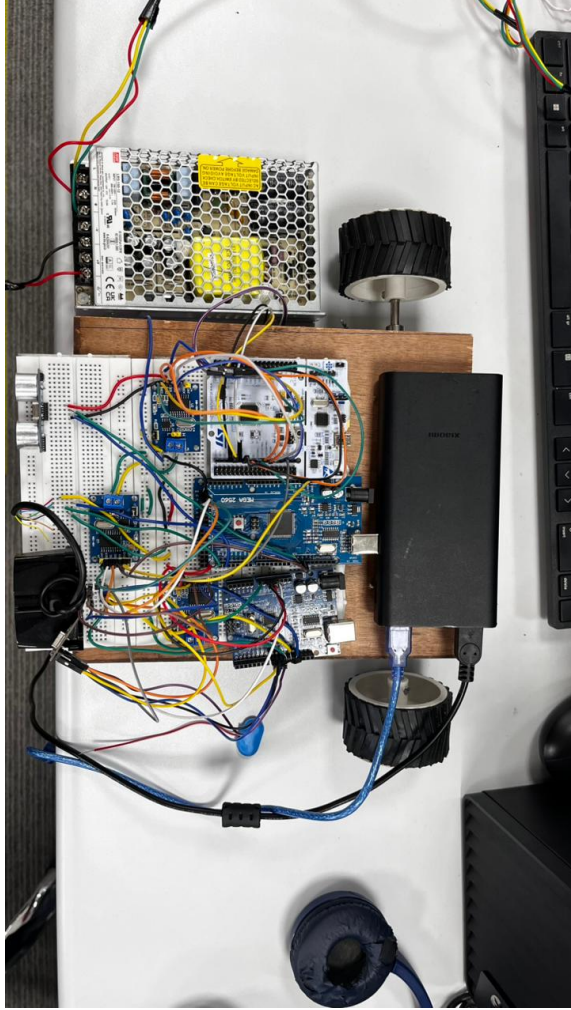


Fig. 6: Top-down view of the test platform used during qualitative and quantitative validation, showing the perception ECU, power distribution, and drive wheels.

- **Sensor suite:** The LIDAR-Lite v3HP and HC-SR04 combination, while adequate for demonstrating fusion principles, lacks the angular resolution and range of automotive-grade radar or solid-state LIDAR, limiting the system to single-point obstacle detection rather than full-scene perception.
- **Bus loading and timing analysis:** The current implementation has not been subjected to formal CAN bus timing/loading analysis (e.g., worst-case latency under full bus load); this would be a natural extension for a more rigorous treatment of the system's real-time guarantees.
- **ACC controller validation:** Full closed-loop stability and robustness analysis of the PID controller (e.g., step-response characterization, disturbance rejection under varying approach speeds) remains future work beyond the qualitative tuning described in Section VI.
- **Scale and actuation:** The DC motor and L298N H-bridge actuation stage is illustrative rather than representative of true vehicle braking dynamics; a future iteration could incorporate a higher-fidelity drivetrain model or a larger-scale RC platform.
- **Formal fault coverage:** The heartbeat deadman-switch covers communication/node-liveness faults but does not yet address other automotive fault classes such as sensor plausibility checking or CAN bus-off recovery handling.

XI. CONCLUSION

This paper presented a multi-node, CAN-bus-based Autonomous Emergency Braking system that reproduces several core architectural patterns of production automotive safety systems on low-cost embedded hardware: distributed sensing and actuation across independent nodes, weighted multi-sensor fusion, a multi-stage braking response, and a hardware-enforced heartbeat fail-safe, with EEPROM-based event data recording, UDS-style diagnostics, and PID-based adaptive cruise control identified as further design targets. The system demonstrates that these patterns can be faithfully integrated, rather than studied in isolation, using Arduino and STM32 development boards. Remaining work centers on instrumented quantitative validation of the timing and control behavior reported theoretically in Section IX-B, and on closing the gap between the ACC and diagnostic design targets in Sections VI and VIII and the firmware verified at the time of writing, as detailed in Appendix D.

REFERENCES

- [1] International Organization for Standardization, "Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling," ISO 11898-1, 2015.
- [2] International Organization for Standardization, "Road vehicles — Unified diagnostic services (UDS)," ISO 14229-1, 2020.
- [3] Microchip Technology Inc., "MCP2515 Stand-Alone CAN Controller with SPI Interface, Datasheet."
- [4] Garmin (PulsedLight), "LIDAR-Lite v3HP Operating Manual and Technical Specifications," Garmin Ltd.
- [5] STMicroelectronics, "STM32F446xC/E Reference Manual," RM0390, STMicroelectronics.
- [6] STMicroelectronics, "NUCLEO-F446RE User Manual," UM1724, STMicroelectronics.
- [7] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: Instrument Society of America, 1995.

APPENDIX

This appendix documents the as-built hardware: the full component inventory, the per-node wiring used on the breadboard prototype, CAN bus wiring across the three CAN-connected nodes, and power distribution. All pin assignments below are taken directly from the verified firmware and the project’s wiring reference rather than reconstructed from the paper narrative.

A. Component Inventory

TABLE III: Full Component Inventory

Component	Role	Detail
Arduino Nano	RF Transmitter	Reads joystick, sends via nRF24L01
HW504 Joystick	Input	VRx → A0, VRy → A1
nRF24L01 #1	Wireless TX	CE=D9, CSN=D10, channel 76, 250 kbps, 3.3 V
Arduino Uno	RF→CAN Bridge	Receives RF, packs to CAN 0x100
nRF24L01 #2	Wireless RX	CE=D9, CSN=D10, same channel and address
MCP2515 #1 (Uno)	CAN Node	CS=D8, 500 kbps, 8 MHz crystal
Arduino Mega 2560	AEB Perception ECU + EDR	LIDAR + HC-SR04 fusion → CAN 0x050 + 0x200; hosts EDR
LIDAR-Lite v3HP	Primary Sensor	I2C, SDA=D20, SCL=D21
HC-SR04	Secondary Sensor	TRIG=D9, ECHO=D8
MCP2515 #2 (Mega)	CAN Node	CS=D10, SCK=D52, SI=D51, SO=D50
MCP2515 #3 (STM32)	CAN Node	CS=D10 (PB6), shared SPI1
STM32 Nucleo F446RE	AEB+RC Controller	Receives 0x100 RC + 0x050/0x200 AEB; originates nothing
L298N	Motor Driver	ENA=D3, IN1=D4, IN2=D5, ENB=D6, IN3=D7, IN4=D8
DC Geared Motors ×2	Actuators	12 V geared DC motors

B. Arduino Nano — RF Transmitter

TABLE IV: Arduino Nano ↔ nRF24L01 #1

From	Pin	To (Nano)	Notes
nRF24L01	VCC	3.3 V	3.3 V only — 5 V destroys the module
nRF24L01	GND	GND	
nRF24L01	CE	D9	Chip Enable
nRF24L01	CSN	D10	SPI chip select
nRF24L01	SCK	D13	SPI clock
nRF24L01	MOSI	D11	SPI data in
nRF24L01	MISO	D12	SPI data out
nRF24L01	IRQ	—	Not used

TABLE V: HW504 Joystick ↔ Arduino Nano

From	Pin	To (Nano)	Notes
HW504	VCC (+5V)	5 V	
HW504	GND	GND	
HW504	VRx	A0	Left/right axis
HW504	VRy	A1	Forward/back axis
HW504	SW	D2	Button — optional, unused in firmware

nRF24L01 Voltage — Critical

nRF24L01 VCC must be 3.3 V. Supplying 5 V permanently destroys the module. Adding a 100 μ F + 100 nF capacitor directly across VCC/GND resolves the majority of intermittent connectivity issues observed during bring-up.

Shared SPI Bus on the Uno

The nRF24L01 (CSN=D10) and the MCP2515 (CS=D8) share the SCK/MOSI/MISO lines on D13/D11/D12. Each device has its own chip-select pin, so no bus conflict occurs as long as the two CS lines remain distinct.

TABLE VI: Arduino Uno ↔ nRF24L01 #2

From	Pin	To (Uno)	Notes
nRF24L01	VCC	3.3 V	3.3 V only
nRF24L01	GND	GND	
nRF24L01	CE	D9	Chip Enable
nRF24L01	CSN	D10	SPI CS (nRF)
nRF24L01	SCK	D13	Shared SPI clock
nRF24L01	MOSI	D11	Shared SPI MOSI
nRF24L01	MISO	D12	Shared SPI MISO

TABLE VII: MCP2515 #1 ↔ Arduino Uno

From	Pin	To (Uno)	Notes
MCP2515	VCC	5 V	
MCP2515	GND	GND	
MCP2515	CS	D8	SPI CS (CAN) — distinct from D10
MCP2515	SCK	D13	Shared SPI clock
MCP2515	SI/MOSI	D11	Shared SPI MOSI
MCP2515	SO/MISO	D12	Shared SPI MISO
MCP2515	INT	D2	Interrupt — unused, polling mode

Arduino Mega — Perception ECU

The Mega fuses LIDAR and ultrasonic distance and broadcasts both AEB telemetry and the system heartbeat on the shared CAN bus. CAN ID 0x050 carries a 1-byte heartbeat (0xA5) every 30 ms; the STM32 watchdog fires after 80 ms of silence. CAN ID 0x200 carries 8-byte distance telemetry every 40 ms. MCP2515 #2 on the Mega shares the CAN bus with MCP2515 #1 (Uno) and MCP2515 #3 (STM32) — all three CANH lines and all three CANL lines must be wired together.

TABLE VIII: MCP2515 #2 ↔ Arduino Mega

From	Pin	To (Mega)	Notes
MCP2515	VCC	5 V	
MCP2515	GND	GND	
MCP2515	CS	D10	SPI chip select
MCP2515	SCK	D52	Mega hardware SPI clock
MCP2515	SI/MOSI	D51	Mega hardware SPI MOSI
MCP2515	SO/MISO	D50	Mega hardware SPI MISO
MCP2515	INT	D2	Optional — not used in polling mode

TABLE IX: LIDAR-Lite v3HP ↔ Arduino Mega

From	Pin	To (Mega)	Notes
LIDAR	VCC (red)	5 V	
LIDAR	GND (black)	GND	
LIDAR	SDA (blue)	D20	I2C data — Mega hardware I2C
LIDAR	SCL (green)	D21	I2C clock
LIDAR	orange wire	—	Not connected
LIDAR	yellow wire	—	Not connected

TABLE X: HC-SR04 ↔ Arduino Mega

From	Pin	To (Mega)	Notes
HC-SR04	VCC	5 V	
HC-SR04	GND	GND	
HC-SR04	TRIG	D9	Trigger output
HC-SR04	ECHO	D8	Echo input

E. STM32 Nucleo F446RE — AEB+RC Motor Controller

TABLE XI: MCP2515 #3 ↔ STM32 Nucleo F446RE

From	Pin	To (Nucleo)	Notes
MCP2515	VCC	5 V	
MCP2515	GND	GND	
MCP2515	CS	D10 (PB6)	SPI chip select
MCP2515	SCK	D13 (PA5)	SPI1 clock
MCP2515	SI/MOSI	D11 (PA7)	SPI1 MOSI
MCP2515	SO/MISO	D12 (PA6)	SPI1 MISO
MCP2515	INT	D2 (PA10)	Optional — polling mode used

TABLE XII: STM32 Nucleo F446RE ↔ L298N Motor Driver

From (Nucleo)	Pin	To (L298N)	Notes
D3 (PB3)	ENA	Motor A speed	PWM (TIM2_CH2)
D4 (PB5)	IN1	Motor A direction	
D5 (PB4)	IN2	Motor A direction	
D6 (PB10)	ENB	Motor B speed	PWM (TIM2_CH3)
D7 (PA8)	IN3	Motor B direction	
D8 (PA9)	IN4	Motor B direction	
GND	—	GND	Common ground — essential

TABLE XIII: L298N Motor Driver ↔ DC Motors

From (L298N)	To	Terminal	Notes
OUT1	Motor A	+	
OUT2	Motor A	—	Swap to reverse direction
OUT3	Motor B	+	
OUT4	Motor B	—	Swap to reverse direction
VIN/12V	12 V PSU	+	Motor power supply, 7–35 V
GND	12 V PSU	—	Also ties to all system grounds

AEB Override Behavior on the STM32

CRUISE (> 500 mm): joystick RC active, forward speed capped at 70% PWM (179/255). DECEL (250–500 mm): linear interpolation ramp, $\text{PWM\%} = 30 + 40 \times (\text{fused} - 250)/250$; joystick steers while AEB governs the speed cap from 70% down to 30%. EMERG (< 250 mm): active back-EMF brake (IN1=IN2=HIGH, ENA=255); joystick completely ignored, zero freewheeling. STATE_NETWORK_FAULT (heartbeat silent > 80 ms): the STM32 enters an infinite hardware lockout loop; the motor stays braked via back-EMF and all 0x200 telemetry is silently discarded — reconnecting the Mega does *not* release the lockout. Recovery requires a debounced press of the PC13 user button (50 ms software debounce on top of the ISR-captured edge timestamp), after which the system returns to STATE_IDLE and re-validates a fresh heartbeat before AEB zones reactivate.

F. CAN Bus Wiring

Three-Node CAN Bus

All three MCP2515 modules (Uno, Mega, STM32) connect to the *same* physical CAN bus: wire CANH of all three together, and CANL of all three together. A 120 Ω termination resistor is required at each physical end of the bus — typically across the Uno module and the STM32 module, with the Mega sitting electrically in the middle. All three boards must also share a common ground for CAN signalling to work correctly.

TABLE XIV: CAN Bus Wiring — All Three Nodes

Node A	Node B	Notes
MCP2515 #1 (Uno) CANH	MCP2515 #2 (Mega) CANH	Wire all CANH together
MCP2515 #2 (Mega) CANH	MCP2515 #3 (STM32) CANH	Same wire
MCP2515 #1 (Uno) CANL	MCP2515 #2 (Mega) CANL	Wire all CANL together
MCP2515 #2 (Mega) CANL	MCP2515 #3 (STM32) CANL	Same wire

TABLE XV: CAN ID Routing Summary

Node	Dir.	ID(s)	Payload
Arduino Uno	TX	0x100	4 bytes — joystick X+Y to STM32
Arduino Mega	TX	0x050	1 byte — heartbeat 0xA5 every 30 ms
Arduino Mega	TX	0x200	8 bytes — AEB distance telemetry every 40 ms
STM32	RX	0x100, 0x050, 0x200	Receives all three; originates none

CAN Bus Settings

Bus speed is 500 kbps (`CAN_500KBPS`) and must be identical across all three `MCP_CAN.begin()` calls. The crystal frequency constant used is `MCP_8MHZ`; check the crystal marking on each physical module and switch to `MCP_16MHZ` uniformly if CAN initialization fails repeatedly. Hardware acceptance filters are disabled on the STM32 (`MCP_ANY` mode, masks and filters zeroed) — message routing is done entirely in software by checking the received identifier.

G. Power Distribution

TABLE XVI: Power Distribution

From	Pin	To	Notes
12 V supply #1	+	L298N VIN/12V	Drives both DC motors
12 V supply #1	—	L298N GND	Common system ground
12 V supply #2	+	Arduino Mega Vin	Powers perception ECU directly
12 V supply #2	—	Arduino Mega GND	
12 V supply #2	+	Arduino Uno Vin	Powers RF-to-CAN bridge directly
12 V supply #2	—	Arduino Uno GND	
5 V-rated supply [†]	+	STM32 VDD pin	[†] Measured output 4.2 V — see warning below
5 V-rated supply [†]	—	STM32 GND	
—	—	Extension board / power strip	Both bench supplies share a common mains extension board
USB	—	Nano USB	Transmitter — or 9 V battery

Hardware Risk — STM32 VDD Overvoltage

The as-built system supplies approximately 4.2 V directly to the STM32 Nucleo F446RE's VDD pin. VDD is the board's 3.3 V logic supply rail; per the STM32F446xC/E datasheet [5], its normal operating range is 1.7–3.6 V and its *absolute maximum rating* is 4.0 V. A continuous 4.2 V supply on VDD exceeds even the absolute maximum rating, not merely the recommended operating range, and carries a real risk of permanent damage to the microcontroller (e.g. latch-up or dielectric breakdown of internal circuitry). This is recorded here as an as-built fact of the current prototype rather than a recommended practice; a safer alternative is to feed an external ~5 V supply into the Nucleo's dedicated 5V pin, which is rated for external 5 V input and feeds the board's onboard regulator down to a safe 3.3 V VDD automatically.

Power Notes

All grounds (Nano, Uno, Mega, STM32, L298N, both bench supplies' negative terminals) must share a common reference. The nRF24L01 VCC must be 3.3 V, with a 100 μ F + 100 nF capacitor across VCC/GND on each module. Both 12 V bench supplies and the STM32's supply are plugged into a shared mains extension board rather than separate wall outlets; this is a convenience for bench testing and does not by itself address the VDD overvoltage risk noted above, since the DC output voltage on the STM32 branch is set independently of which extension board it is plugged into. Remove the L298N's 12 V jumper on ENA/ENB — the firmware drives these pins directly via PWM. A 100 μ F capacitor across each motor terminal reduces back-EMF noise coupling onto the CAN bus.

This appendix presents the verified firmware for all four nodes, in full, as flashed to the hardware. Listings 1 and 2 (Nano transmitter and Uno bridge) are unchanged from the original RC control system that predates the AEB extension. Listing 3 is the current perception ECU, including the EEPROM-based EDR. Listing 4 is the current motor controller; as noted in Section VI and Section VIII of the main paper, this revision implements the Decel zone as a linear interpolation ramp rather than the full PID control law, and does not yet implement the UDS-style diagnostic layer — both are tracked in Appendix D.

H. Arduino Nano — Joystick Transmitter

The Nano reads the two joystick potentiometer axes on A0/A1 and transmits them as a packed JoyData struct over a 2.4 GHz nRF24L01 link using the RF24 library, on channel 76 at 250 kbps with the radio held in low transmit power. No CAN hardware is present on this node — it is purely a wireless front end for the joystick.

```
1  /* =====
2  * TRANSMITTER - Arduino Nano
3  * Hardware : Arduino Nano + HW504 Joystick + nRF24L01
4  * Library  : RF24 by TMRh20 (install via Library Manager)
5  * UNCHANGED from original system
6  * ===== */
7
8  #include <SPI.h>
9  #include <RF24.h>
10
11 // nRF24L01: CE=D9, CSN=D10 (hardware SPI on D11/D12/D13)
12 RF24 radio(9, 10);
13
14 const byte ADDR[6] = "MOTOR"; // Must match Uno receiver
15
16 struct JoyData {
17     uint16_t x; // VRx analog value (0-1023)
18     uint16_t y; // VRY analog value (0-1023)
19 };
20
21 void setup() {
22     Serial.begin(9600);
23     if (!radio.begin()) {
24         Serial.println("ERROR: nRF24L01 not detected! Check wiring.");
25         while (true);
26     }
27     radio.setChannel(76);
28     radio.setPALevel(RF24_PA_LOW);
29     radio.setDataRate(RF24_250KBPS);
30     radio.openWritingPipe(ADDR);
31     radio.stopListening();
32     Serial.println("Transmitter ready.");
33 }
34
35 void loop() {
36     JoyData data;
37     data.x = analogRead(A0);
38     data.y = analogRead(A1);
39     bool ok = radio.write(&data, sizeof(data));
40     Serial.print("X: "); Serial.print(data.x);
41     Serial.print(" Y: "); Serial.print(data.y);
42     Serial.println(ok ? " [SENT OK]" : " [SEND FAILED]");
43     delay(20);
44 }
```

Listing 1: Arduino Nano — RF joystick transmitter

I. Arduino Uno — RF-to-CAN Bridge

The Uno listens for the joystick packet on the same nRF24L01 channel and address, then immediately repackages the two 16-bit axis values into a 4-byte big-endian CAN frame and transmits it on the shared bus under identifier 0x100 via a second

MCP2515 module (chip-select D8, distinct from the radio's CSN on D10). This node performs no AEB logic; it is a pure protocol bridge.

```
1  /* =====
2  * RECEIVER / CAN BRIDGE - Arduino Uno
3  * Hardware : Uno + nRF24L01 (RX) + MCP2515 (CAN TX)
4  * UNCHANGED from original system
5  *
6  * nRF24L01 : CE=D9, CSN=D10
7  * MCP2515 : CS=D8 (shared SPI D11/D12/D13)
8  * Sends CAN ID 0x100 - joystick data to STM32
9  * ===== */
10
11 #include <SPI.h>
12 #include <RF24.h>
13 #include <mcp_can.h>
14
15 RF24 radio(9, 10);
16 MCP_CAN CAN(8);
17
18 const byte ADDR[6] = "MOTOR";
19 const long CAN_ID = 0x100;
20
21 struct JoyData { uint16_t x; uint16_t y; };
22
23 void setup() {
24   Serial.begin(9600);
25   if (!radio.begin()) {
26     Serial.println("ERROR: nRF24L01 not detected!");
27     while (true);
28   }
29   radio.setChannel(76);
30   radio.setPALevel(RF24_PA_LOW);
31   radio.setDataRate(RF24_250KBPS);
32   radio.openReadingPipe(0, ADDR);
33   radio.startListening();
34   Serial.println("nRF24L01 ready (RX mode)");
35
36   while (CAN.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) != CAN_OK) {
37     Serial.println("MCP2515 init failed - check wiring / crystal freq.");
38     delay(300);
39   }
40   CAN.setMode(MCP_NORMAL);
41   Serial.println("MCP2515 CAN ready (TX node)");
42 }
43
44 void loop() {
45   if (!radio.available()) return;
46   JoyData data;
47   radio.read(&data, sizeof(data));
48   byte buf[4] = {
49     (byte)(data.x >> 8), (byte)(data.x & 0xFF),
50     (byte)(data.y >> 8), (byte)(data.y & 0xFF)
51   };
52   byte result = CAN.sendMsgBuf(CAN_ID, 0, 4, buf);
53   Serial.print("RF<- X="); Serial.print(data.x);
54   Serial.print(" Y="); Serial.print(data.y);
55   Serial.println(result == CAN_OK ? " -> CAN OK" : " -> CAN FAIL");
56 }
```

Listing 2: Arduino Uno — RF-to-CAN bridge

J. Arduino Mega 2560 — AEB Perception ECU and EDR

The Mega is the most complex node in the system. Each loop iteration polls the LIDAR-Lite v3HP over I2C and the HC-SR04 over a trigger/echo pulse pair, fuses the two readings, applies a moving-average filter, transmits both the heartbeat and the 8-byte telemetry frame, and feeds the result into the EDR state machine.

Sensor fusion (weightedFuse): both readings are range-validated independently (lValid/uValid). If only one sensor is in range, that reading is used directly and the system is flagged as fusion-degraded. If both disagree by more than DISAGREEMENT_MM (300 mm) for FAULT_FRAMES (5) consecutive samples, the conservative (closer) reading is used and the fusion-degraded flag is set. Otherwise, a zone-dependent weighted average is computed — 25% LIDAR weight below 250 mm, 60% between 250–1000 mm, and 90% beyond 1000 mm — with the weighted sum computed in long arithmetic specifically to avoid the 16-bit overflow defect discussed in Appendix D.

Heartbeat priority injection (lidarReady): the LIDAR ready-poll can block for up to 80 ms. To guarantee the STM32's 80 ms watchdog is never starved by this blocking call, the polling loop itself checks elapsed time since the last heartbeat

transmission and injects a heartbeat frame whenever 30 ms have passed, making heartbeat transmission the de facto highest-priority task on this node.

Event Data Recorder (edr_sample, edr_flush, edr_dump_serial, edr_reset): a 10-frame ring buffer continuously records pre-trigger telemetry at 20 Hz. When the fused distance first drops below the 250 mm emergency threshold, the EDR transitions to EDR_TRIGGERED and begins appending the trigger frame plus 10 post-trigger frames to a separate buffer. Once all 21 frames (10 pre + 1 trigger + 10 post, 12 bytes each, 252 bytes of data plus a 2-byte header for a 254-byte EEPROM footprint) are collected, edr_flush writes them to internal EEPROM, injecting a heartbeat after every frame write (each frame's 12 sequential EEPROM.update() calls take roughly 40 ms at 3.3 ms/byte) so the STM32 watchdog is serviced throughout the entire ≈ 832 ms worst-case write. Captured data is retrieved post-test by connecting over USB and sending 'D' (CSV dump) or 'R' (clear and re-arm) over the serial monitor.

```

1  /* =====
2  * AEB PERCEPTION ECU + EDR - Arduino Mega 2560
3  * Hardware : Mega + LIDAR-Lite v3HP (I2C) + HC-SR04 + MCP2515
4  * Library  : MCP_CAN by coryjfowler
5  *
6  * NEW - EDR (Event Data Recorder), internal EEPROM, no SD card
7  * 10 pre-crash + 1 trigger + 10 post-crash frames @ 20 Hz
8  * Trigger : fused_mm transitions below 250 mm (EMERGENCY zone onset)
9  * Storage : 21 frames x 12 bytes = 252 bytes (of 4096 EEPROM available)
10 * Readback : connect USB after test, open Serial Monitor 115200 baud
11 *          send 'D' -> CSV dump | send 'R' -> clear + re-arm
12 *
13 * Transmits on same CAN bus as Uno and STM32:
14 * ID 0x050 Heartbeat 1 byte 0xA5 every 30 ms
15 * ID 0x200 AEB telem 8 bytes every 40 ms
16 *
17 * CAN frame 0x200:
18 * Byte 0-1 Fused distance mm (uint16 big-endian)
19 * Byte 2-3 Raw LIDAR mm
20 * Byte 4-5 Raw Ultrasonic mm
21 * Byte 6 Status Bit0=LIDAR_OK Bit1=US_OK Bit2=DEGRADED
22 * Byte 7 Rolling counter 0-255
23 *
24 * MCP2515 : CS=D10 SCK=D52 SI=D51 SO=D50
25 * LIDAR : SDA=D20 SCL=D21
26 * HC-SR04 : TRIG=D9 ECHO=D8
27 * ===== */
28
29 #include <Wire.h>
30 #include <SPI.h>
31 #include <mcp_can.h>
32 #include <EEPROM.h>
33
34 MCP_CAN CAN(10);
35
36 const long CAN_HB_ID = 0x050;
37 const long CAN_AEB_ID = 0x200;
38
39 #define LIDAR_ADDR 0x62
40 #define TRIG_PIN 9
41 #define ECHO_PIN 8
42
43 #define LIDAR_MIN_MM 50
44 #define LIDAR_MAX_MM 10000
45 #define ULTRA_MIN_MM 20
46 #define ULTRA_MAX_MM 4000
47 #define ZONE_NEAR_MAX 250
48 #define ZONE_MID_MAX 1000
49 #define DISAGREEMENT_MM 300
50 #define FAULT_FRAMES 5
51
52 // EMERGENCY zone threshold - mirrors STM32 zone logic
53 #define EMERG_THRESHOLD_MM 250
54
55 int disagreementCount = 0;
56 bool fusionDegraded = false;
57
58 #define FILTER_SIZE 5
59 int filterBuf[FILTER_SIZE];
60 int filterIdx = 0;
61 long filterSum = 0;
62 bool filterReady = false;
63
64 unsigned long lastHBTx = 0;
65 unsigned long lastTelTx = 0;
66 uint8_t rollingCtr = 0;
67
68 // =====

```

```

69 // EDR - Event Data Recorder (Internal EEPROM, no SD card)
70 //
71 // Frame layout : 10 PRE + 1 TRIGGER + 10 POST = 21 frames
72 // Frame size   : 12 bytes (packed struct, no padding on AVR)
73 // EEPROM used  : 2 header bytes + 252 data bytes = 254 bytes total
74 //
75 // EEPROM address map:
76 //   [0]          0xED = valid data / 0xFF = empty
77 //   [1]          number of frames written (0-21)
78 //   [2-253]      frame array (21 x 12 bytes)
79 //
80 // Write timing:
81 //   252 bytes x 3.3 ms/byte ~= 832 ms worst-case.
82 //   A heartbeat (0xA5 on CAN 0x050) is injected after writing each
83 //   frame (~40 ms per frame). This keeps the STM32 80 ms watchdog
84 //   alive throughout the entire EEPROM write sequence.
85 //
86 // Post-test readback (Serial 115200 baud, USB connected):
87 //   'D' or 'd' -> dump all frames as CSV to Serial Monitor
88 //   'R' or 'r' -> clear EEPROM, re-arm EDR for next test
89 // =====
90
91 // -- Frame struct - 12 bytes on AVR -----
92 typedef struct {
93     uint32_t timestamp_ms; // millis() at capture time
94     int16_t  lidar_mm;     // LIDAR-Lite v3HP raw reading (mm)
95     int16_t  sonar_mm;     // HC-SR04 raw reading (mm)
96     int16_t  fused_mm;     // weighted sensor fusion output (mm)
97     uint8_t  sensor_status; // Bit0=LIDAR_OK Bit1=US_OK Bit2=DEGRADED
98     uint8_t  phase;        // 0 = PRE 1 = TRIGGER 2 = POST
99 } EDR_Frame_t;
100
101 // -- Sizing constants -----
102 #define EDR_PRE_COUNT 10
103 #define EDR_POST_COUNT 10
104 // edr_post[] holds the TRIGGER frame (slot 0) + 10 POST frames (slots 1-10)
105 #define EDR_POST_SLOTS (EDR_POST_COUNT + 1) // 11
106 #define EDR_TOTAL_FRAMES (EDR_PRE_COUNT + EDR_POST_SLOTS) // 21
107 #define EDR_FRAME_SIZE ((uint8_t)sizeof(EDR_Frame_t)) // 12
108 #define EDR_SAMPLE_MS 50 // 20 Hz sampling rate
109
110 // -- EEPROM address constants -----
111 #define EDR_ADDR_MARKER 0
112 #define EDR_ADDR_COUNT 1
113 #define EDR_ADDR_DATA 2
114 #define EDR_VALID_MARKER 0xED
115
116 // -- SRAM buffers -----
117 static EDR_Frame_t edr_ring[EDR_PRE_COUNT];
118 static uint8_t edr_ring_head = 0;
119 static uint8_t edr_ring_count = 0;
120
121 static EDR_Frame_t edr_post[EDR_POST_SLOTS];
122 static uint8_t edr_post_count = 0;
123
124 // -- EDR state machine -----
125 typedef enum {
126     EDR_IDLE, // recording pre-crash ring buffer continuously
127     EDR_TRIGGERED, // EMERGENCY detected - collecting post-crash frames
128     EDR_SAVED // all 21 frames written to EEPROM
129 } EDR_State_t;
130
131 static EDR_State_t edr_state = EDR_IDLE;
132 static uint32_t edr_last_ms = 0;
133 static bool edr_was_emerg = false;
134
135 // -- Private: fill one EDR frame -----
136 static void edr_pack(EDR_Frame_t *f, int lidar, int sonar,
137                     int fused, uint8_t status, uint8_t phase) {
138     f->timestamp_ms = millis();
139     f->lidar_mm = (int16_t)constrain(lidar, -32768, 32767);
140     f->sonar_mm = (int16_t)constrain(sonar, -32768, 32767);
141     f->fused_mm = (int16_t)constrain(fused, -32768, 32767);
142     f->sensor_status = status;
143     f->phase = phase;
144 }
145
146 // -- Private: write all frames to EEPROM with HB injection -----
147 static void edr_flush() {
148     EDR_Frame_t all[EDR_TOTAL_FRAMES];
149     uint8_t idx = 0;
150     uint8_t start = (edr_ring_count < EDR_PRE_COUNT) ? 0 : edr_ring_head;
151     for (uint8_t i = 0; i < edr_ring_count; i++)

```

```

152     all[idx++] = edr_ring[(start + i) % EDR_PRE_COUNT];
153     for (uint8_t i = 0; i < edr_post_count; i++)
154         all[idx++] = edr_post[i];
155     uint8_t total = idx;
156
157     EEPROM.update(EDR_ADDR_MARKER, EDR_VALID_MARKER);
158     EEPROM.update(EDR_ADDR_COUNT, total);
159
160     int addr = EDR_ADDR_DATA;
161     for (uint8_t i = 0; i < total; i++) {
162         const uint8_t *p = (const uint8_t *)&all[i];
163         for (uint8_t b = 0; b < EDR_FRAME_SIZE; b++)
164             EEPROM.update(addr++, p[b]);
165         // HB injection after each frame (~40 ms) - keeps STM32 80 ms watchdog alive
166         if (millis() - lastHBTx >= 30) {
167             byte hb[1] = {0xA5};
168             CAN.sendMsgBuf(CAN_HB_ID, 0, 1, hb);
169             lastHBTx = millis();
170         }
171     }
172     edr_state = EDR_SAVED;
173     Serial.print(F("[EDR] Write complete - "));
174     Serial.print(total);
175     Serial.println(F(" frames saved. Send 'D' to dump."));
176 }
177
178 // -- Public: edr_sample() - call every loop iteration -----
179 void edr_sample(int lidar, int sonar, int fused,
180                uint8_t status, bool is_emerg) {
181     if (edr_state == EDR_SAVED) return;
182     uint32_t now = millis();
183     if ((now - edr_last_ms) < EDR_SAMPLE_MS) return;
184     edr_last_ms = now;
185
186     if (edr_state == EDR_IDLE) {
187         edr_pack(&edr_ring[edr_ring_head], lidar, sonar, fused, status, 0);
188         edr_ring_head = (edr_ring_head + 1) % EDR_PRE_COUNT;
189         if (edr_ring_count < EDR_PRE_COUNT) edr_ring_count++;
190         if (is_emerg && !edr_was_emerg) {
191             edr_state = EDR_TRIGGERED;
192             edr_pack(&edr_post[edr_post_count++], lidar, sonar, fused, status, 1);
193             Serial.println(F("[EDR] CRASH TRIGGER - capturing post-crash frames..."));
194         }
195         edr_was_emerg = is_emerg;
196     } else if (edr_state == EDR_TRIGGERED) {
197         if (edr_post_count < EDR_POST_SLOTS)
198             edr_pack(&edr_post[edr_post_count++], lidar, sonar, fused, status, 2);
199         if (edr_post_count >= EDR_POST_SLOTS)
200             edr_flush();
201     }
202 }
203
204 // -- Public: edr_dump_serial() - print CSV -----
205 void edr_dump_serial() {
206     if (EEPROM.read(EDR_ADDR_MARKER) != EDR_VALID_MARKER) {
207         Serial.println(F("[EDR] No crash log in EEPROM. Run a test first.)); return;
208     }
209     uint8_t count = EEPROM.read(EDR_ADDR_COUNT);
210     Serial.println(F("==== EDR CRASH LOG ====="));
211     Serial.println(F("phase,timestamp_ms,lidar_mm,sonar_mm,fused_mm,sensor_status"));
212     int addr = EDR_ADDR_DATA;
213     for (uint8_t i = 0; i < count; i++) {
214         EDR_Frame_t f;
215         uint8_t *p = (uint8_t *)&f;
216         for (uint8_t b = 0; b < EDR_FRAME_SIZE; b++) p[b] = EEPROM.read(addr++);
217         const char *ph = (f.phase==0)?"PRE":(f.phase==1)?"TRIGGER":"POST";
218         Serial.print(ph);          Serial.print(',');
219         Serial.print(f.timestamp_ms); Serial.print(',');
220         Serial.print(f.lidar_mm);     Serial.print(',');
221         Serial.print(f.sonar_mm);      Serial.print(',');
222         Serial.print(f.fused_mm);      Serial.print(',');
223         Serial.println(f.sensor_status, BIN);
224     }
225     Serial.println(F("==== END OF LOG ====="));
226 }
227
228 // -- Public: edr_reset() - clear + re-arm -----
229 void edr_reset() {
230     EEPROM.update(EDR_ADDR_MARKER, 0xFF);
231     EEPROM.update(EDR_ADDR_COUNT, 0);
232     memset(edr_ring, 0, sizeof(edr_ring));
233     memset(edr_post, 0, sizeof(edr_post));
234     edr_ring_head=0; edr_ring_count=0; edr_post_count=0;

```

```

235     edr_state=EDR_IDLE; edr_was_emerg=false;
236     Serial.println(F("[EDR] Cleared. Armed for next test."));
237 }
238
239 // =====
240 // SENSOR + FUSION - original Mega code, unchanged
241 // =====
242
243 void lidarTrigger() {
244     Wire.beginTransmission(LIDAR_ADDR); Wire.write(0x00); Wire.write(0x04); Wire.endTransmission();
245 }
246 bool lidarReady(uint16_t ms=80) {
247     // HB-PRIORITY NOTE: LIDAR polling can block up to ~ms milliseconds.
248     // Heartbeat injected inside this loop whenever 30 ms have elapsed.
249     unsigned long t=millis();
250     while (millis()-t<ms) {
251         Wire.beginTransmission(LIDAR_ADDR); Wire.write(0x01); Wire.endTransmission(false);
252         Wire.requestFrom(LIDAR_ADDR,1);
253         if (Wire.available() && !(Wire.read()&0x01)) return true;
254         if (millis()-lastHBTx >= 30) {
255             byte hb[1]={0xA5}; CAN.sendMsgBuf(CAN_HB_ID,0,1,hb); lastHBTx=millis();
256         }
257         delayMicroseconds(500);
258     } return false;
259 }
260 int lidarReadMM() {
261     Wire.beginTransmission(LIDAR_ADDR); Wire.write(0x0F|0x80); Wire.endTransmission(false);
262     Wire.requestFrom(LIDAR_ADDR,2);
263     if (Wire.available()>=2) { uint8_t h=Wire.read(),l=Wire.read(); return(int) ((h<<8)|l)*10; }
264     return -1;
265 }
266 int ultraReadMM() {
267     digitalWrite(TRIG_PIN,LOW); delayMicroseconds(2);
268     digitalWrite(TRIG_PIN,HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN,LOW);
269     long d=pulseIn(ECHO_PIN,HIGH,30000UL);
270     return d==0?-1:(int) (d*0.1715f);
271 }
272 bool lValid(int d){return d>=LIDAR_MIN_MM&&d<=LIDAR_MAX_MM;}
273 bool uValid(int d){return d>=ULTRA_MIN_MM&&d<=ULTRA_MAX_MM;}
274
275 int weightedFuse(int lidar,int ultra){
276     bool lv=lValid(lidar),uv=uValid(ultra);
277     if(!lv&&!uv){fusionDegraded=true;disagreementCount=0;return 0;}
278     if(lv&&!uv){fusionDegraded=true;disagreementCount=0;return lidar;}
279     if(!lv&&uv){fusionDegraded=true;disagreementCount=0;return ultra;}
280     if(abs(lidar-ultra)>DISAGREEMENT_MM){
281         if(++disagreementCount>=FAULT_FRAMES) fusionDegraded=true;
282         return min(lidar,ultra);
283     }
284     disagreementCount=0;fusionDegraded=false;
285     int r=min(lidar,ultra);
286     int wL=(r<ZONE_NEAR_MAX)?25:(r<=ZONE_MID_MAX)?60:90;
287     return(int) ((Long) (wL*lidar+(100-wL)*ultra)/100L);
288 }
289
290 void filterInit(int v){for(int i=0;i<FILTER_SIZE;i++)filterBuf[i]=v;filterSum=(long)v*FILTER_SIZE;filterReady=true;}
291 int filterApply(int v){filterSum+=filterBuf[filterIdx];filterBuf[filterIdx]=v;filterSum+=v;filterIdx=(filterIdx+1)%
    FILTER_SIZE;return(int) (filterSum/FILTER_SIZE);}
292
293 void sendHeartbeat(){
294     if(millis()-lastHBTx<30) return; lastHBTx=millis();
295     byte d[1]={0xA5}; CAN.sendMsgBuf(CAN_HB_ID,0,1,d);
296 }
297 void sendTelemetry(int fused,int lidar,int ultra,uint8_t status){
298     if(millis()-lastTelTx<40) return; lastTelTx=millis();
299     uint16_t pF=(uint16_t)max(0,fused),pL=(uint16_t)max(0,lidar),pU=(uint16_t)max(0,ultra);
300     byte f[8]={highByte(pF),lowByte(pF),highByte(pL),lowByte(pL),highByte(pU),lowByte(pU),status,rollingCtr++};
301     CAN.sendMsgBuf(CAN_AEB_ID,0,8,f);
302 }
303
304 // =====
305 // SETUP
306 // =====
307
308 void setup(){
309     Serial.begin(115200); Wire.begin();
310     pinMode(TRIG_PIN,OUTPUT); pinMode(ECHO_PIN,INPUT); digitalWrite(TRIG_PIN,LOW);
311     pinMode(10,OUTPUT); digitalWrite(10,HIGH);
312     while (CAN.begin(MCP_ANY,CAN_500KBPS,MCP_8MHZ)!=CAN_OK){Serial.println("CAN init failed...");delay(200);}
313     CAN.setMode(MCP_NORMAL);
314     // EDR startup: alert if previous crash log still in EEPROM
315     if(EEPROM.read(EDR_ADDR_MARKER)==EDR_VALID_MARKER){
316         Serial.println(F("[EDR] !! Unread crash log in EEPROM.));

```

```

317     Serial.println(F("[EDR]    Send 'D' to dump CSV | 'R' to clear + re-arm.));
318 } else {
319     Serial.println(F("[EDR] Armed. Trigger: fused_mm < 250.));
320 }
321 Serial.println("AEB Perception ECU + EDR ready. TX 0x050 HB + 0x200 AEB.");
322 }
323
324 // =====
325 // LOOP
326 // =====
327
328 void loop() {
329     sendHeartbeat();
330     lidarTrigger(); int rawL=lidarReady()?lidarReadMM():-1;
331     sendHeartbeat();
332     int rawU=ultraReadMM();
333     int fused=weightedFuse(rawL,rawU); if(fused<0)fused=0;
334     if(!filterReady&&fused>0)filterInit(fused);
335     int final_d=filterReady?filterApply(fused):fused; if(final_d<0)final_d=0;
336     uint8_t status=0;
337     if(lValid(rawL))status|=0x01; if(uValid(rawU))status|=0x02; if(fusionDegraded)status|=0x04;
338     sendTelemetry(final_d,max(0,rawL),max(0,rawU),status);
339
340     // EDR: trigger only when fused > 0 (real obstacle, not sensor failure)
341     bool is_emerg = (final_d > 0) && (final_d < EMERG_THRESHOLD_MM);
342     edr_sample(rawL, rawU, final_d, status, is_emerg);
343
344     // Post-test Serial commands (connect USB after run)
345     if(Serial.available()){
346         char c=Serial.read();
347         if(c=='D' || c=='d') edr_dump_serial();
348         if(c=='R' || c=='r') edr_reset();
349     }
350 }

```

Listing 3: Arduino Mega 2560 — AEB perception ECU with EEPROM-based EDR

K. STM32 Nucleo F446RE — AEB+RC Motor Controller

The STM32 is the only node that arbitrates between human input and safety override. Every loop iteration drains the full MCP2515 receive FIFO, routing each frame by CAN identifier: 0x100 updates the joystick axes and re-derives motor PWM under the current AEB zone’s authority; 0x050 simply timestamps the most recent heartbeat; 0x200 updates fusedMM and re-evaluates the zone state machine (CRUISE / DECEL / EMERG).

Zone arbitration: in CRUISE or IDLE, the joystick’s combined differential-drive output is clamped to $\pm$ CRUISE_PWM (179, i.e. 70%). In DECEL, a linear interpolation ($\text{pct} = 30 + 40 \cdot (\text{fusedMM} - 250) / 250$) computes a PWM cap that ramps from 70% at 500 mm down to 30% at 250 mm, and the joystick output is clamped to that cap. In EMERG or FAULT, the joystick branch is not evaluated at all — the motors are driven exclusively by `brakeAll()` elsewhere in the loop.

Hardware lockout (STATE_NETWORK_FAULT): once a heartbeat has been seen at least once and then goes silent for AEB_WATCHDOG_MS (80 ms), the firmware enters a `while(true)` loop that is not part of the normal control flow — it can only be exited by a debounced button press. Inside the loop, `brakeAll()` is re-asserted every iteration, the CAN FIFO is still drained (to avoid MCP2515 buffer overflow and to log recovered heartbeats), and the LED flashes at 10 Hz. The button ISR (`onButton`) fires on the falling edge of PC13 and records `millis()` into `btnPressTime`; the main loop only honors the press once `millis() - btnPressTime ≥ 50` ms, filtering mechanical contact bounce. Clearing the lockout resets the heartbeat-tracking state entirely, forcing a fresh heartbeat to be observed before AEB zones can reactivate.

```

1  /* =====
2  * AEB + RC MOTOR CONTROLLER - STM32 Nucleo-64 F446RE
3  * Hardware : Nucleo F446RE + MCP2515 + L298N + 2x DC motors
4  *
5  * CAN inputs (all on same 500 kbps bus):
6  *   0x100 Joystick data   from Arduino Uno   (RC control)
7  *   0x050 AEB heartbeat   from Arduino Mega   (watchdog)
8  *   0x200 AEB distance     from Arduino Mega   (zones + telemetry)
9  *
10 * AEB PRIORITY - governs motor speed limit by zone:
11 *   > 500 mm CRUISE       Joystick active, speed capped at 70% (179/255 PWM)
12 *   250-500mm DECEL       Linear ramp: PWM% = 30 + (40*(fused-250)/250)
13 *                         70% at 500mm -> 30% at 250mm (raw 178->76)
14 *   < 250 mm EMERG        Active back-EMF brake (IN1=IN2=HIGH), RC ignored
15 *   HB>80ms STATE_NETWORK_FAULT Hardware lockout loop - all 0x200 ignored,
16 *                         10Hz LED, motor locked. Only exit: PC13 User button
17 *                         with mandatory 50ms software debounce.
18 *
19 * MCP2515: CS=D10 (PB6)  SCK=D13 (PA5)  MOSI=D11 (PA7)  MISO=D12 (PA6)
20 * Motor A: ENA=D3 (PB3)  IN1=D4 (PB5)   IN2=D5 (PB4)

```



```

21  * Motor B: ENB=D6 (PB10) IN3=D7 (PA8) IN4=D8 (PA9)
22  * ===== */
23
24 #include <SPI.h>
25 #include <mcp_can.h>
26
27 MCP_CAN CAN(10);
28
29 const long CAN_RC_ID = 0x100;
30 const long CAN_HB_ID = 0x050;
31 const long CAN_AEB_ID = 0x200;
32
33 #define ENA 3
34 #define IN1 4
35 #define IN2 5
36 #define ENB 6
37 #define IN3 7
38 #define IN4 8
39
40 const int DEADZONE = 30;
41 const int CRUISE_PWM = 179;
42
43 const unsigned long RC_WATCHDOG_MS = 500;
44 const unsigned long AEB_WATCHDOG_MS = 80;
45
46 unsigned long lastRCmsg = 0, lastHBmsg = 0;
47 bool hbEverRx = false;
48 uint16_t fusedMM = 9999;
49
50 typedef enum { AEB_IDLE, AEB_CRUISE, AEB_DECEL, AEB_EMERG, AEB_FAULT } AEBState;
51 AEBState aebState = AEB_IDLE;
52
53 volatile bool btnFlag = false;
54 volatile unsigned long btnPressTime = 0;
55
56 void onButton() {
57     btnPressTime = millis();
58     btnFlag = true;
59 }
60
61 void driveMotorA(int spd) {
62     spd = constrain(spd, -255, 255);
63     digitalWrite(IN1, spd > 0 ? HIGH : LOW);
64     digitalWrite(IN2, spd < 0 ? HIGH : LOW);
65     analogWrite(ENA, abs(spd));
66 }
67 void driveMotorB(int spd) {
68     spd = constrain(spd, -255, 255);
69     digitalWrite(IN3, spd > 0 ? HIGH : LOW);
70     digitalWrite(IN4, spd < 0 ? HIGH : LOW);
71     analogWrite(ENB, abs(spd));
72 }
73 void stopAll() { driveMotorA(0); driveMotorB(0); }
74 void brakeAll() {
75     analogWrite(ENA, 255); digitalWrite(IN1, HIGH); digitalWrite(IN2, HIGH);
76     analogWrite(ENB, 255); digitalWrite(IN3, HIGH); digitalWrite(IN4, HIGH);
77 }
78
79 void setup() {
80     Serial.begin(115200);
81     pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
82     pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
83     stopAll();
84     pinMode(PC13, INPUT_PULLUP);
85     attachInterrupt(digitalPinToInterrupt(PC13), onButton, FALLING);
86
87     while(CAN.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) != CAN_OK){
88         Serial.println("MCP2515 init failed - retrying..."); delay(300);
89     }
90     CAN.setMode(MCP_NORMAL);
91     CAN.init_Mask(0, 0, 0x000); CAN.init_Mask(1, 0, 0x000);
92     for(int i = 0; i < 6; i++) CAN.init_Filt(i, 0, 0x000);
93
94     Serial.println("AEB+RC Motor Controller ready.");
95     Serial.println("IDs: 0x100=RC 0x050=HB 0x200=AEB");
96 }
97
98 void loop() {
99     while(CAN.checkReceive() == CAN_MSGAVAIL){
100         unsigned long id; byte len, buf[8];
101         CAN.readMsgBuf(&id, &len, buf);
102
103         if(id == CAN_RC_ID && len >= 4){

```

```

104     lastRCmsg = millis();
105     uint16_t rawX = ((uint16_t)buf[0] << 8) | buf[1];
106     uint16_t rawY = ((uint16_t)buf[2] << 8) | buf[3];
107     int xVal = map((int)rawX, 0, 1023, -255, 255);
108     int yVal = map((int)rawY, 0, 1023, -255, 255);
109     if(abs(xVal) < DEADZONE) xVal = 0;
110     if(abs(yVal) < DEADZONE) yVal = 0;
111
112     if(aebState == AEB_CRUISE || aebState == AEB_IDLE){
113         driveMotorA(constrain(yVal + xVal, -CRUISE_PWM, CRUISE_PWM));
114         driveMotorB(constrain(yVal - xVal, -CRUISE_PWM, CRUISE_PWM));
115     } else if(aebState == AEB_DECEL){
116         int pct = 30 + (40 * (int)(fusedMM - 250)) / 250;
117         pct = constrain(pct, 30, 70);
118         int cap = (pct * 255) / 100;
119         driveMotorA(constrain(yVal + xVal, -cap, cap));
120         driveMotorB(constrain(yVal - xVal, -cap, cap));
121     }
122 }
123 else if(id == CAN_HB_ID && len >= 1 && buf[0] == 0xA5){
124     lastHBmsg = millis(); hbEverRx = true;
125 }
126 else if(id == CAN_AEB_ID && len >= 8 && aebState != AEB_FAULT){
127     fusedMM = ((uint16_t)buf[0] << 8) | buf[1];
128     if(fusedMM < 250) aebState = AEB_EMERG;
129     else if(fusedMM <= 500) aebState = AEB_DECEL;
130     else aebState = AEB_CRUISE;
131 }
132 }
133
134 if(hbEverRx && (millis() - lastHBmsg) >= AEB_WATCHDOG_MS){
135     if(aebState != AEB_FAULT){
136         aebState = AEB_FAULT;
137         Serial.println("!! NETWORK FAULT - hardware lockout engaged");
138         while(true) {
139             brakeAll();
140             digitalWrite(LED_BUILTIN, (millis() % 100 < 50) ? HIGH : LOW);
141             while(CAN.checkReceive() == CAN_MSGAVAIL){
142                 unsigned long rxId; byte rxLen; rxBuf[8];
143                 CAN.readMsgBuf(&rxId, &rxLen, rxBuf);
144                 if(rxId == CAN_HB_ID && rxLen >= 1 && rxBuf[0] == 0xA5)
145                     Serial.println("[LOCKOUT] HB rx - motor locked until button reset");
146             }
147             if(btnFlag && (millis() - btnPressTime) >= 50UL){
148                 btnFlag=false; btnPressTime=0;
149                 aebState=AEB_IDLE; hbEverRx=false; lastHBmsg=0; lastRCmsg=0;
150                 stopAll();
151                 Serial.println("[BTN] Lockout cleared -> STATE_IDLE");
152                 break;
153             }
154         }
155     }
156 }
157
158 if(lastRCmsg > 0 && millis() - lastRCmsg > RC_WATCHDOG_MS)
159     if(aebState == AEB_CRUISE || aebState == AEB_IDLE) stopAll();
160
161 if(aebState == AEB_EMERG || aebState == AEB_FAULT) brakeAll();
162
163 if(aebState == AEB_FAULT || aebState == AEB_EMERG)
164     digitalWrite(LED_BUILTIN, (millis() % 100 < 50) ? HIGH : LOW);
165 else if(aebState == AEB_DECEL)
166     digitalWrite(LED_BUILTIN, (millis() % 500 < 250) ? HIGH : LOW);
167 else
168     digitalWrite(LED_BUILTIN, HIGH);
169 }

```

Listing 4: STM32 Nucleo F446RE — AEB+RC motor controller

Calibration and Tuning Notes

If motors creep at rest, increase DEADZONE (default 30). If a motor spins the wrong direction, swap its IN1/IN2 (or IN3/IN4) wires at the L298N terminal rather than in firmware. The AEB watchdog (80 ms) is set against a 30 ms Mega heartbeat interval, giving roughly 2.5× margin. The DECEL interpolation range (30%–70%) is currently a fixed pair of constants in the pct formula; changing the minimum braking floor means editing the literal 30 values directly.

This appendix documents the test procedures used to validate the heartbeat fail-safe and lockout mechanism described in Section V-B, along with the timing budget and CAN bus load analysis that underpin them. These procedures target the

STM32 motor controller and Mega perception ECU specifically; ACC and diagnostic-layer test procedures are not yet defined, consistent with the implementation status noted in Appendix D.

L. Watchdog Timing Parameters

TABLE XVII: Watchdog Timing Parameters

Parameter	Value
Heartbeat transmit interval (Mega)	30 ms
Watchdog threshold (STM32)	80 ms
Frame margin before FAULT	≈ 2.67 frames
Spec-required minimum margin	2.5 frames
LIDAR polling timeout	80 ms
HB injection inside <code>lidarReady()</code>	$\geq$ every 30.5 ms
Worst-case HB gap (with LIDAR)	$30 \text{ ms} + 500 \mu\text{s} \approx 30.5 \text{ ms}$
Back-EMF brake assert latency	$< 1 \text{ loop @ } 180 \text{ MHz} \approx \mu\text{s}$
Button debounce period	50 ms
Typical button bounce duration	$< 20 \text{ ms}$
Debounce margin over bounce	$\geq 2.5\times$
CAN bus speed	500 kbps
0x050 frame duration on bus	$\approx 44 \mu\text{s}$
0x200 frame duration on bus	$\approx 154 \mu\text{s}$

M. Pre-Test Safety Checklist

Pre-Test Safety Checklist

- Power the L298N motor supply from a bench PSU with a current limit set to $\leq 2 \text{ A}$ — prevents runaway if braking fails.
- Lift the vehicle on a block so drive wheels spin freely — never run the bus-sever test on a surface without containment.
- Have a PCAN-View session open on ID filter 0x050 and 0x200 before pulling wires — live timestamps are needed for the timing proof.
- Confirm the STM32 Serial monitor is open at 115200 baud — the lockout print statements are essential for timing analysis.
- Run each test three times and record all three results. Report the worst-case timing observed, not the average.

N. Test 1 — Bus-Sever Cut Test

Goal: prove that physically severing the CAN-H wire halts the motor via `STATE_NETWORK_FAULT` within the 80 ms watchdog window.

Procedure:

- 1) Flash all three boards. Verify CRUISE state (motor spinning, Serial shows "`>500mm CRUISE`").
- 2) Open PCAN-View. Start timestamp logging on ID 0x050. Note the last received frame timestamp.
- 3) With one sharp pull, remove the CAN-H jumper wire from the Mega MCP2515 breadboard terminal.
- 4) Observe the motor. Start a stopwatch on the pull.
- 5) Record the STM32 Serial output — note the timestamp of the "`!! NETWORK FAULT`" print.
- 6) In PCAN-View, the last 0x050 frame is T_{last} . Note the STM32 wall-clock time of lockout entry.
- 7) Repeat three times. Record the worst-case elapsed time between the last heartbeat frame and lockout.

Timing proof: motor halts $\leq 80 \text{ ms}$ after the last valid 0x050 frame. $T_{\text{fault}} = T_{\text{last_HB}} + \Delta_{\text{HB_watchdog}}$ (80 ms). The STM32 loop runs at 180 MHz $\approx$ nanosecond granularity — actual latency is dominated by the watchdog poll interval, not loop overhead.

Pass criteria:

- Motor stops with zero freewheeling (back-EMF brake engaged, not coast-stop).
- STM32 Serial shows "`!! NETWORK FAULT -- HB silent >80ms`" within the window.
- LED begins 10 Hz flashing within the same window.
- All three trials halt within 80 ms of the last observed 0x050 frame.

O. Test 2 — Crash Verification (Mega Reset Hold)

Goal: simulate a Mega microcontroller crash. The STM32 must detect the silence and lock identically to Test 1.

Procedure:

- 1) Reconnect CAN-H. Verify normal CRUISE operation.
- 2) Locate the physical RESET button on the Arduino Mega board.
- 3) Hold RESET down continuously — the Mega halts; the CAN-H/L lines go passive (recessive).
- 4) Observe the motor and the STM32 LED.
- 5) Release RESET after 2 seconds. Verify the Mega resumes heartbeat transmission.
- 6) Confirm the STM32 stays in `STATE_NETWORK_FAULT` — the resumed heartbeat must *not* auto-clear the lockout.
- 7) Press the Nucleo PC13 user button (hold ≥ 50 ms). Verify the system returns to `STATE_IDLE`.

Timing proof: the same 80 ms watchdog applies. A Mega RESET causes immediate CAN silence with no graceful shutdown frame; PCAN-View will show the 0x050 stream cutting off mid-sequence.

Pass criteria:

- Motor locks within 80 ms of the RESET press.
- LED flashes at 10 Hz.
- When the Mega is released and heartbeat resumes, STM32 Serial prints "[LOCKOUT] HB rx -- motor locked until button reset".
- Motor remains stopped with heartbeat active — proving the lockout cannot be cleared by network recovery alone.
- PC13 press clears the lockout after the 50 ms debounce; Serial prints "[BTN] Lockout cleared -> STATE_IDLE".

P. Test 3 — Lockout Override Test

Goal: verify the hardware lockout loop correctly ignores all 0x200 AEB telemetry and 0x100 joystick frames while in `STATE_NETWORK_FAULT`.

Procedure:

- 1) Trigger `STATE_NETWORK_FAULT` via the bus-sever method (Test 1). Confirm the motor is locked.
- 2) Reconnect CAN-H. In PCAN-View, confirm 0x050 and 0x200 frames are visible again.
- 3) Move the joystick to full forward. Verify the motor stays completely stopped.
- 4) Send a 0x200 frame with fused distance > 500 mm (CRUISE zone) — motor must not move.
- 5) Observe STM32 Serial — "[LOCKOUT] HB rx" confirms frames are received but discarded.
- 6) Press the Nucleo PC13 user button. Time the debounce, holding for exactly 50 ms minimum.
- 7) Verify Serial prints "[BTN] Lockout cleared -> STATE_IDLE".
- 8) Verify the system returns to normal CRUISE operation with joystick control.

Timing proof: the 50 ms debounce is measured from the falling edge captured by the `onButton()` ISR (`btnPresTime = millis()`). Elapsed time is `millis() - btnPresTime`. At 180 MHz, `millis()` resolution is 1 ms, giving debounce accuracy of ± 1 ms.

Pass criteria:

- Motor stays stopped when the network recovers — 0x200 telemetry does not release the lockout.
- Joystick (0x100) has no effect on motor output during lockout.
- A button press shorter than 50 ms (simulated contact bounce) does *not* clear the lockout.
- A button press ≥ 50 ms clears the lockout reliably on all three trial runs.
- After clearing, AEB zones (CRUISE/DECEL/EMERG) reactivate normally on the next 0x200 frame.

Q. CAN Bus Load Analysis

TABLE XVIII: CAN Bus Load Analysis (500 kbps)

Frame	DLC	Interval	Duration	Load
0x050	1 B	30 ms	$\approx 44 \mu\text{s}$	0.15%
0x100	4 B	20 ms	$\approx 111 \mu\text{s}$	0.56%
0x200	8 B	40 ms	$\approx 154 \mu\text{s}$	0.39%
Total bus load			$\approx 1.10\%$ (spec $\leq 40\%$)	

Frame duration is computed as $(1 \text{ start} + 11 \text{ ID} + 6 \text{ ctrl} + 8 \times \text{DLC data} + 15 \text{ CRC} + 3 \text{ EOF} + 7 \text{ IFS}) \text{ bits} \div 500,000 \text{ bps}$. At an aggregate load of roughly 1.1% against a 40% design budget, the current three-frame traffic pattern leaves substantial headroom for the diagnostic request/response traffic described in Section VIII once implemented.

This appendix records the project’s development history as a chronology of concrete issues encountered and resolved, plus a transparent account of the open items identified while preparing this documentation.

R. Note on Source Provenance

The hardware and firmware details in Appendices A and B were verified directly against the project’s interactive firmware/wiring reference application and the most recently flashed source files, rather than reconstructed from the main paper’s narrative. During that verification, the heartbeat-origin claim in the main paper (Section III-A and Table I) was found to be inconsistent with the firmware: the paper originally stated that the STM32 originates the 0x050 heartbeat, while the verified firmware shows the heartbeat being transmitted by the Arduino Mega’s `sendHeartbeat()` function and the `lidarReady()` priority-injection path, with the STM32 acting purely as a consumer. This has been corrected throughout the main paper (Section III-A, Section V-B, Table I, and Figure 1) rather than left as a known discrepancy, since it is a factual question about which node owns which CAN identifier rather than a matter of design intent.

S. SPI Bus Contention and Chip-Select Conflicts

As described in Section III-C, the distributed architecture required two SPI-bus devices — the nRF24L01 radio and the MCP2515 CAN controller — to coexist on the same physical SPI bus (SCK/MOSI/MISO) on both the Uno and, in earlier prototyping, alongside an SD card breakout on the perception node. The resolution applied throughout the verified firmware is consistent chip-select discipline: each device retains its own dedicated CS/CSN pin (e.g., the Uno’s nRF24L01 uses CSN=D10 while its MCP2515 uses CS=D8), with no two devices sharing a chip-select line. This is visible directly in Appendix A’s per-node wiring tables and is the practical embodiment of the bus-arbitration fix described narratively in the main paper.

T. Integer-Overflow Fix in the Sensor Fusion Math

Section IV-B of the main paper describes a 16-bit intermediate-overflow defect in the weighted-average fusion calculation, resolved by widening the relevant intermediate variables to 32-bit signed integers. This fix is directly visible in the verified `weightedFuse()` function (Appendix B, Listing 3):

```
int wL = (r < ZONE_NEAR_MAX) ? 25 : (r <= ZONE_MID_MAX) ? 60 : 90;
return (int)((long)(wL * lidar + (100 - wL) * ultra) / 100L);
```

The cast of the first operand to `long` before the multiplication forces the entire expression into 32-bit arithmetic, which is the documented fix. Without this cast, `wL * lidar` is evaluated in the native 16-bit `int` width on the AVR-based Mega, and can overflow when `lidar` exceeds roughly 700 mm at the higher weight values — silently wrapping to a negative or otherwise incorrect intermediate sum.

U. EDR Write Timing Versus the Heartbeat Watchdog

Section VII-C describes the risk of a naive blocking EEPROM write starving the heartbeat long enough to trip the STM32’s 80 ms watchdog during an EDR capture event. The verified `edr_flush()` routine (Appendix B, Listing 3) confirms the fix described there: a heartbeat transmission is attempted after every completed 12-byte frame write (each frame taking roughly 40 ms at the AVR’s ~3.3 ms-per-byte EEPROM write time), keeping the gap between heartbeats safely under the 80 ms threshold throughout the full ~832 ms worst-case write of all 21 frames. This timing relationship is also why the EDR sampling rate (20 Hz, i.e. one sample every 50 ms) and the heartbeat interval (30 ms) were chosen as a compatible pair rather than independently: the EDR’s own sampling cadence is slower than the heartbeat interval, so normal sampling never competes with heartbeat delivery, and only the bulk EEPROM flush at the end of a capture needed the explicit injection logic.

V. Open Issue: ACC Controller Divergence from the PID Design

Section VI of the main paper specifies a discrete-time PID control law (Eq. 1) with derivative low-pass filtering, targeting a 300 mm following gap. The STM32 firmware verified while preparing this documentation (Appendix B, Listing 4) does not implement this control law: the Decel zone response is a linear interpolation ramp on the PWM duty-cycle cap, driven directly by the raw fused-distance reading with no explicit proportional, integral, or derivative terms, and no 300 mm setpoint. The two designs are not equivalent — the linear ramp caps speed as a function of absolute distance, whereas the PID design regulates speed as a function of the *error* between distance and a fixed target gap.

This is recorded here as an open item rather than resolved in the main paper text, per the decision to treat ACC and diagnostics as future work for this revision of the documentation. Two non-exclusive explanations are consistent with the available evidence: either the PID controller exists in a firmware revision not included in the source files reviewed for this documentation pass, or the linear-ramp implementation represents a deliberate simplification adopted during tuning that has not yet been reflected back into the main paper’s Section VI. Resolving which is the case — and, if the PID controller is real, locating and verifying its source — is the first follow-up item for the next documentation pass.

W. Open Issue: UDS-Style Diagnostic Layer Not Yet Implemented

Section VIII describes a request/response diagnostic service layer modeled on ISO 14229. No corresponding CAN identifier, request parser, or response builder appears in any of the four firmware files reviewed for this documentation (Appendix B). The CAN bus load analysis in Appendix C confirms there is ample headroom (under 2% utilization against a 40% budget) to add this traffic without a redesign of bus timing. This remains an unimplemented design target rather than a verified subsystem.

X. Open Issue: STM32 VDD Overvoltage Risk

During the power-distribution review for this documentation pass, it was confirmed that the STM32 Nucleo F446RE's VDD pin is supplied directly from a bench power supply nominally rated 5 V but measured at approximately 4.2 V, rather than from the board's dedicated 5V input or its onboard regulator. As detailed in Appendix A, this exceeds the STM32F446xC/E's absolute maximum VDD rating of 4.0 V [5]. This is recorded as an open hardware risk rather than resolved here, since correcting it requires a physical rewiring change (moving the supply connection from VDD to the board's 5V pin) rather than a firmware or documentation fix. No functional failures attributable to this overvoltage condition have been observed to date, but absence of observed failure is not evidence of safety at this margin above the absolute maximum rating, and the wiring should be corrected before further extended operation.